

READING SHEET

Context API & State Management

Module 3 · Frontend: React & Next.js · Day 32 · Week 7

PROGRAM

CodeOps · Full Stack

WORK TYPE

Revision + mini-project

EST. TIME

2–3 hours

Tip How to use this sheet

This is your take-away for Day 32. It revises the context patterns worth knowing, explains what a state library actually adds, teaches Zustand as the tool you will build with, and shows enough Redux Toolkit to read it confidently in an existing codebase. The Addis Eats cart moves out of a provider and into a store — and almost nothing in your screens has to change.

Part 1 · Context in Depth

You already use context for the cart: a reducer owning the transitions, a memoised value, and any component reading it in one line. For an application the size of Addis Eats that is a perfectly good answer. This part is about using it *well* — and being honest about where it stops.

Wrap the context in its own hook

```
export function useCart() {
  const ctx = useContext(CartContext);
  if (ctx === null) {
    throw new Error("useCart must be used inside a CartProvider");
  }
  return ctx;
}
```

useCart.js — a friendlier front door

Without the guard, forgetting the provider gives you null and a crash somewhere unrelated. The hook fails immediately and says exactly what is wrong. Export the hook and keep the context object itself private to the module — every serious React codebase does it this way.

Split contexts by how often they change

```
<AppContext.Provider value={{ user, cart, theme, language }}>

// add one dish -> every consumer of user, theme and language
// re-renders too, for no reason at all

// better:
<AuthProvider><ThemeProvider><CartProvider>
```

one context for everything re-renders everything

The theme changes almost never; the cart changes on every click. Keeping them in one object ties them together so the rare value pays the cost of the busy one. Splitting **state** from **dispatch** is another useful division: dispatch never changes, so components that only write never re-render when the data does.

Where context stops being enough

CONTEXT HANDLES IT	YOU WANT A LIBRARY
A handful of shared values	Dozens of pieces of shared state
Changes on user actions	Changes many times a second
Every consumer needs the change	Each screen needs a different slice
Reading state in components	Reading state outside React too

Warning The missing piece is selection

Context has no way to say "only re-render me when the total changes" — every consumer gets everything. That is the one capability a store adds. But most applications never outgrow context: adding a library to a small project buys you configuration, not capability.

Part 2 · What a Store Adds

With context, the state lives in a component and exists only because a provider is mounted above you. With a **store**, the state lives in a plain module outside the tree — so a component, an event handler, a route loader or a test can all read it. Components then **subscribe** to the slice they use.

```
// context: I get the whole value, every time it changes
const { items, total, user } = useCart();

// store: I get one slice, and only re-render for that slice
const total = useCartStore(s => s.total);
```

the difference in one line

That small function is the whole idea. It says which part of the state this component depends on, and the library uses it to decide whether a re-render is needed at all.

The vocabulary

TERM	WHAT IT MEANS
store	The object holding all the shared state
selector	A function picking the slice you need
action	A function that changes the state
middleware	Code running on every change — logging, saving
devtools	A browser panel showing every change over time

Redux, Zustand and the rest differ in syntax but share this vocabulary — and you have met most of it already. An action is your reducer's action; subscribing is what useContext does, minus the selection. Middleware is the only genuinely new piece, and it is what gives you persistence and time-travel debugging almost for free.

Choosing for this program

	ZUSTAND	REDUX TOOLKIT
Setup	One function	Store, slices, provider

	ZUSTAND	REDUX TOOLKIT
Boilerplate	Very little	Structured, more of it
Provider needed	No	Yes
Where you meet it	New projects	Existing codebases

You build with **Zustand** because it teaches the ideas with almost no ceremony, and you learn to read **Redux Toolkit** because a great deal of professional React is already written in it. They are not rivals — an interviewer asking about state management wants to hear that you know why a store exists, not which brand you prefer.

Part 3 · Zustand

A store in one function call, with no provider to mount. `create` takes a function receiving `set` and returning the initial state; the actions live in that same object.

```
cartStore.js – the whole store

npm install zustand
import { create } from "zustand";

export const useCartStore = create((set) => ({
  items: [],
  addItem: (dish) =>
    set((state) => ({ items: [...state.items, dish] })),
  clear: () => set({ items: [] })
}));
```

`set` merges rather than replaces, so `set({ items: [] })` leaves every other key untouched. Pass a function when the new value depends on the old one.

Reading with selectors

```
CartBadge.jsx – subscribe to one slice

// re-renders only when items changes
const items = useCartStore((s) => s.items);

// this component only writes – it never re-renders
const addItem = useCartStore((s) => s.addItem);

// avoid: takes the whole store, re-renders on any change
const store = useCartStore();
```

Because the store is a module, any component imports it directly. Nothing has to be mounted above you, and moving a component around the tree cannot break its access to state.

Warning Select narrowly, and mind the references

The selector is the entire benefit — taking the whole store makes Zustand behave exactly like context. And returning a fresh object from a selector defeats the comparison Zustand uses, so it always looks changed. Prefer one selector per value; if you truly need several at once, Zustand ships a shallow comparison helper for that case.

Moving the Addis Eats cart

what changes in the components

```
// before - context
const { items, dispatch } = useCart();
dispatch({ type: "add", dish });

// after - store
const addItem = useCartStore((s) => s.addItem);
addItem(dish);
```

The CartProvider, the reducer file and the useMemo on the provider value all disappear; imports and selectors replace them. Each screen changes one or two lines, which is the sign of a well-separated application. Nothing about Day 31 changes — the cart now survives navigation because it lives outside the tree entirely, not because the layout stays mounted.

Part 4 · Redux Toolkit

You will not build with Redux today, but you should be able to read it — and you already know every idea it is built from. createSlice takes the initial state and the reducers and generates the action creators for you.

cartSlice.js – createSlice

```
const cartSlice = createSlice({
  name: "cart",
  initialState: { items: [] },
  reducers: {
    addItem: (state, action) => {
      state.items.push(action.payload); // looks like mutation
    },
    clear: (state) => { state.items = []; }
  }
});
export const { addItem, clear } = cartSlice.actions;
```

Tip Why push is allowed here

Redux Toolkit uses a library called Immer to record your changes against a draft object and produce a new one from them. You write what looks like mutation; it produces immutability. This applies only inside a slice — everywhere else the Day 28 rule still holds.

the three remaining pieces

```
export const store = configureStore({
  reducer: { cart: cartSlice.reducer } });

<Provider store={store}><App /></Provider>

const items = useSelector((s) => s.cart.items);
const dispatch = useDispatch();
dispatch(addItem(dish));
```

useSelector is the selector you just met in Zustand, dispatch is the dispatch from your own reducer, and the Provider is the context provider you have used since Day 30. Redux asks for more setup and gives back a strict, predictable structure that large teams rely on — that trade is the whole argument between the two.

Choosing well

Start with **useState** — most state belongs in the component using it. Lift to **context** when a few values are shared widely and change on user actions. Reach for a **store** when there are many slices, frequent updates, or a real need for devtools and persistence.

Warning Keep server data out of all three

Putting fetched data in a global store means you now own caching, refetching and staleness by hand. Server data is not really application state — Day 41 gives it a proper home in SWR or React Query. Let the store hold what is genuinely yours: the cart, the session, the interface.

Assigned Reading

Required

1. [Zustand — Getting started](#) What / why: creating a store, selectors, actions and the persist middleware.
2. [Redux Toolkit — Quick start](#) What / why: createSlice, configureStore and the hooks, in about twenty minutes of reading.

Optional — go deeper

- [You Might Not Need Redux](#) — the argument for reaching for a store later rather than sooner.
- [Scaling up with reducer and context](#) — the pattern this day starts from, in full.

Exercises

Work these in order and commit as you go.

1. Wrap CartContext in a useCart hook that throws a clear error when the provider is missing.
2. Split auth and theme into their own providers, separate from the cart provider.
3. Turn on "highlight updates" in React DevTools and record what re-renders when you add a dish.
4. Install zustand and build a cart store with items, addItem, remove and clear.
5. Replace each useCart call with a narrow selector, one component at a time.
6. Add the persist middleware and confirm the order survives a refresh.
7. Rebuild just the cart as a Redux Toolkit slice, without wiring it in, and compare the two files.

Mini-Project · The Cart, Moved to a Store

Tip What you build

Take the routed application from Day 31 and move the cart out of context and into a Zustand store: narrow selectors in every consumer, actions on the store itself, and persistence so an order survives a refresh. The auth session stays in context, now behind a guarded hook — so you finish with a considered reason for each choice.

Requirements

- A useCart-style guarded hook for every context that remains.
- Separate providers for auth and theme, no longer bundled into one value.
- A Zustand cart store holding items with addItem, remove and clear actions.
- Every consumer reading through a narrow selector — no bare useCartStore() calls.

- The persist middleware configured, with the order restored on reload.
- A short README section explaining why the cart is in a store and the session is not.

A worked start

```
starter – store with persistence

import { create } from "zustand";
import { persist } from "zustand/middleware";

export const useCartStore = create(persist((set) => ({
  items: [],
  addItem: (dish) => set((s) => ({ items: [...s.items, dish] })),
  remove: (id) => set((s) => ({
    items: s.items.filter((d) => d.id !== id) })),
  clear: () => set({ items: [] }),
}), { name: "addis-eats-cart" }));
```

What to submit

Your GitHub repository with cartStore.js, useAuth.js and the updated screens, plus the README section on why each piece of state lives where it does.

Check yourself

- Does removing the CartProvider break anything at all?
- Does adding a dish still re-render the header, or only the badge?
- Does the order survive a full page refresh?
- Does every component select one value rather than the whole store?
- Does using a context hook outside its provider throw a message you can act on?
- Can you say in one sentence why the session stayed in context?